PR1 Crossing Equivalence Report

This report summarizes the updated semantics of `pr1-unify-unweighted-search-api`. The search target is the canonical 4-pin crossing gadget with boundary pins `[1, 2, 3, 4]`, and those boundary roles are now kept fixed during search. The purpose of the branch is not to search for isolated extra vertices, but to recognize gadgets that still realize the same crossing even if their crossing arms are stretched by inserted path vertices or realized with logical flips on the boundary pins.

Goal

We want `search_unweighted_gadgets(...)` to find replacements for the canonical crossing target:

- pin `1` connects toward pin `3`
- pin `2` connects toward pin `4`
- the two pin pairs represent the crossing we ultimately want to realize

The updated equivalence class now includes:

- edge subdivision along any target edge, e.g. `1-3` becoming `1-5-3`
- repeated subdivision on both crossing arms, e.g. `1-5-7-3` and `2-6-8-4`
- logical flips on the boundary pins, represented as tensor flips rather than graph rewrites

This means a candidate can match an expanded or flipped form and still be accepted as a match for the original crossing gadget.

What Changed In PR1

- The old isolated-vertex expansion semantics were removed from `equivalent_representations(...)`.
- Target expansion now distributes the added-vertex budget across the target edges and rewrites edges into longer paths using degree-2 inserted vertices.
- Logical-flip patterns are generated and applied at the reduced-alpha-tensor level.
- The existing reduced-alpha-tensor deduplication stage is kept, so duplicate structural or flip variants are collapsed before search.
- The existing filter architecture is kept, so candidate graphs are still checked by inf-mask compatibility and `is_diff_by_constant(...)`.

Search Flow

1. Start from the canonical crossing target and its fixed boundary `[1, 2, 3, 4]`.
2. Generate structural equivalents by subdividing target edges with an explicit `max_added_vertices` expansion budget, while keeping boundary pin roles fixed.
3. Generate logical-flip tensor variants for the resulting target family.
4. Deduplicate all generated representations by their reduced alpha tensors.
5. Precompute cached comparison data for every surviving target variant.
6. Scan candidate graphs from the loader and test candidate boundary choices.
7. Accept a candidate as soon as one target variant matches up to a constant offset, then report it as a replacement for the original crossing target.

Figure 1: Search Pipeline

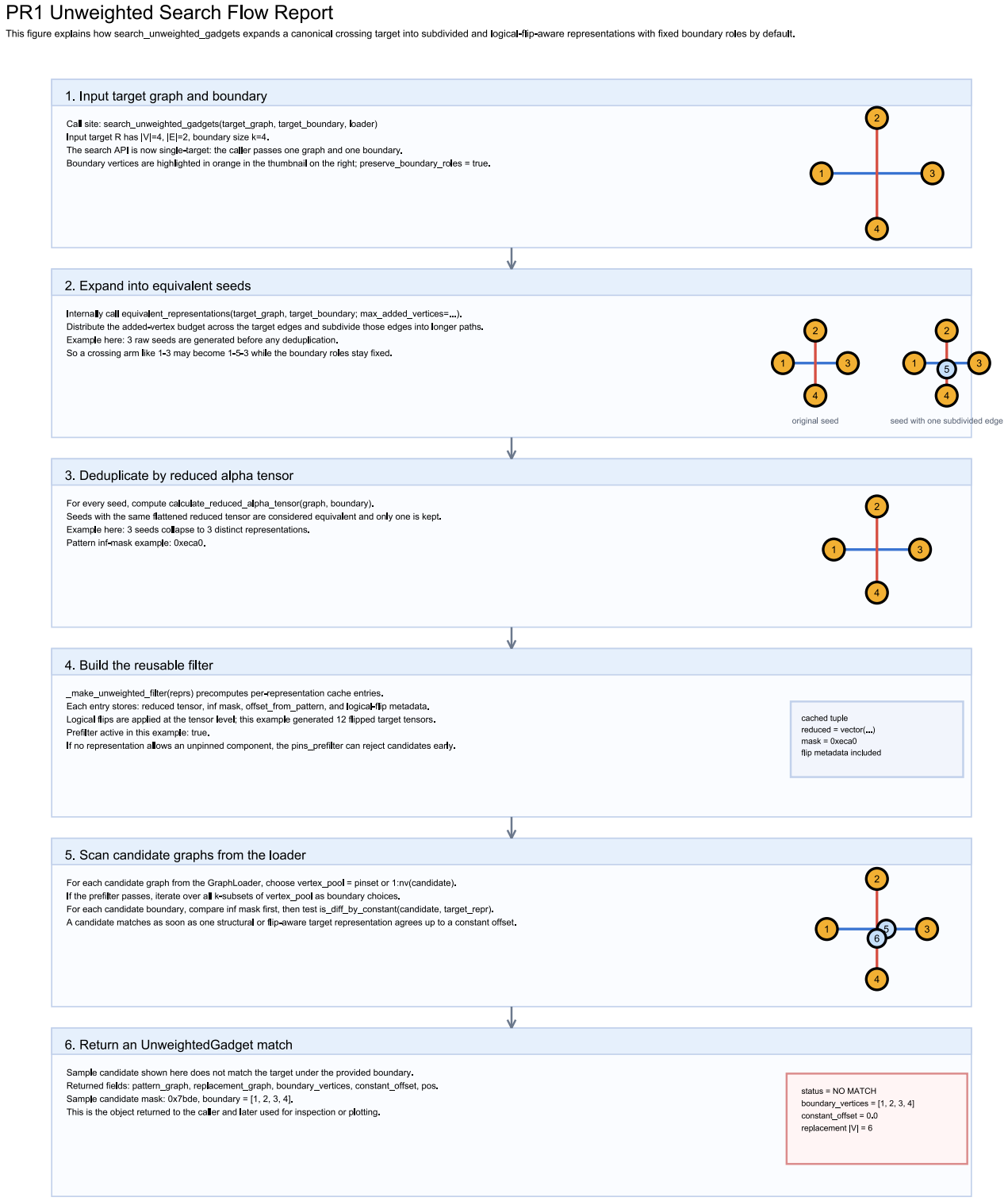

Figure 1: Report-style view of the updated `search_unweighted_gadgets(...)` flow. The important change is that target expansion now means path subdivision on target edges plus logical-flip tensor variants, not isolated-vertex growth.

Figure 2: Example Equivalent Representations

Canonical Crossing Equivalence Gallery

Fixed roles: 1 = left, 2 = top, 3 = right, 4 = bottom. Blue shows the 1-3 arm; red shows the 2-4 arr

Panels show structural examples from equivalent_representations(...); logical flips remain tensor

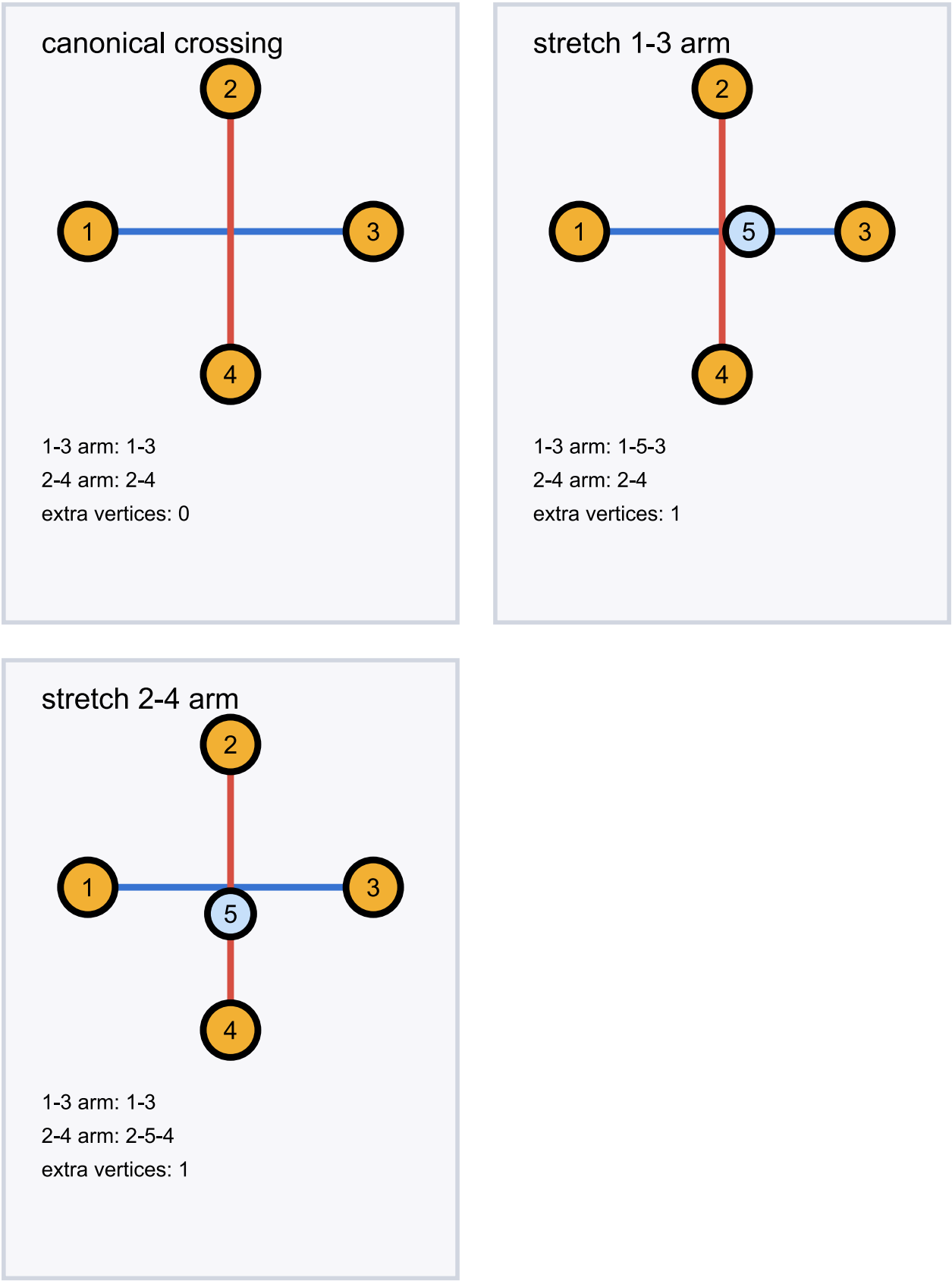

Figure 2: Geometry-fixed gallery for the canonical crossing target. Here the roles are drawn explicitly as `1 = left`, `2 = top`, `3 = right`, and `4 = bottom`. The blue arm tracks the `1-3` channel and the red arm tracks the `2-4` channel, so subdivisions such as `1-5-3` or `2-6-4` still read visually as the same crossing. Logical flips are

Validation Summary

- The unweighted-search tests were rewritten to check subdivision-based equivalence instead of isolated-vertex equivalence.
- Focused tests now cover single-edge subdivision, two-edge subdivision, longer subdivision chains, logical-flip target generation, and successful recovery of matches against the canonical crossing target.
- Visualization tests still pass after updating the report and equivalent representation diagrams.

Bottom Line

The updated PR1 behavior is now aligned with the actual crossing-gadget goal:

- a candidate does not need to look exactly like the smallest crossing graph
- it may realize the same crossing through subdivided target edges
- it may also realize a logical-flip variant that can later be converted back to the canonical crossing semantics
- but it is no longer allowed to satisfy the target only by reordering the boundary labels

So the branch now searches for the crossing gadget through a richer and more useful equivalence class, instead of through isolated-vertex expansions that do not preserve the intended crossing interpretation.